



SÃO
PAULO
TECH
SCHOOL

Pesquisa e Inovação

2 APIs

web-data-viz + dat-acqu-ino

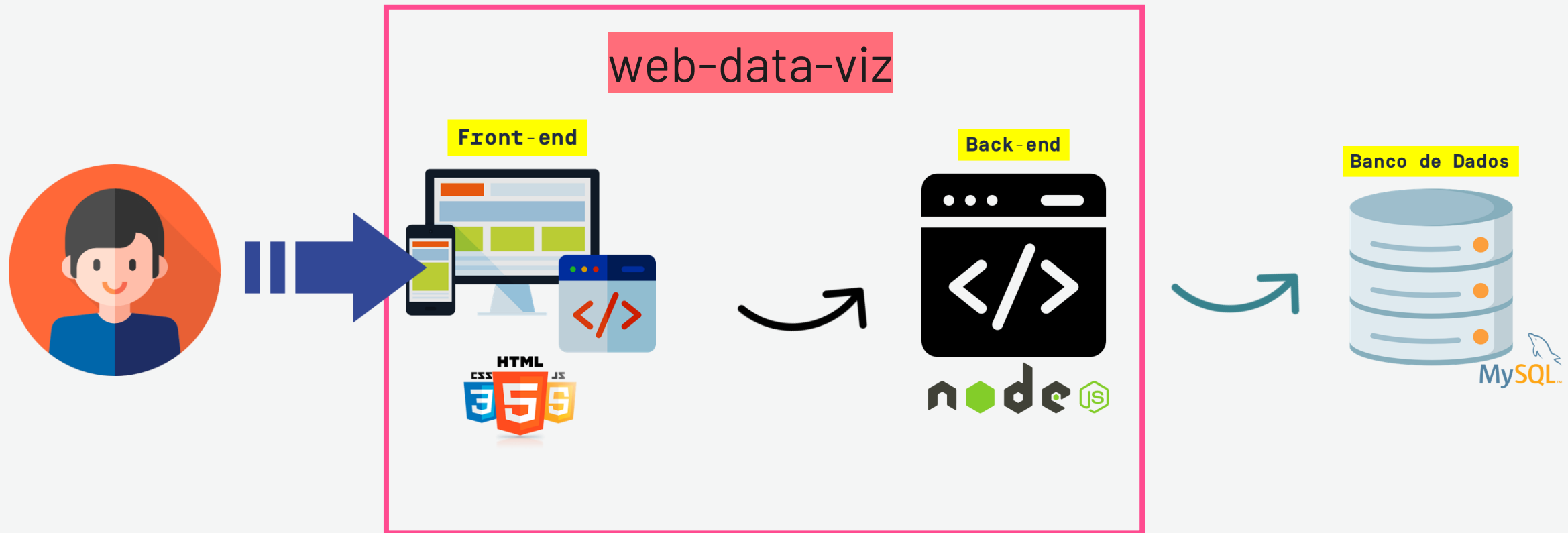
**DROPS DE CONTEÚDO PARA
ENTENDER AS ESTRUTURAS
“NOVAS” PRESENTES NO CÓDIGO DO
WEB-DATA-VIZ**

Vimos tudo isso para quê?

2 APIs!!!!

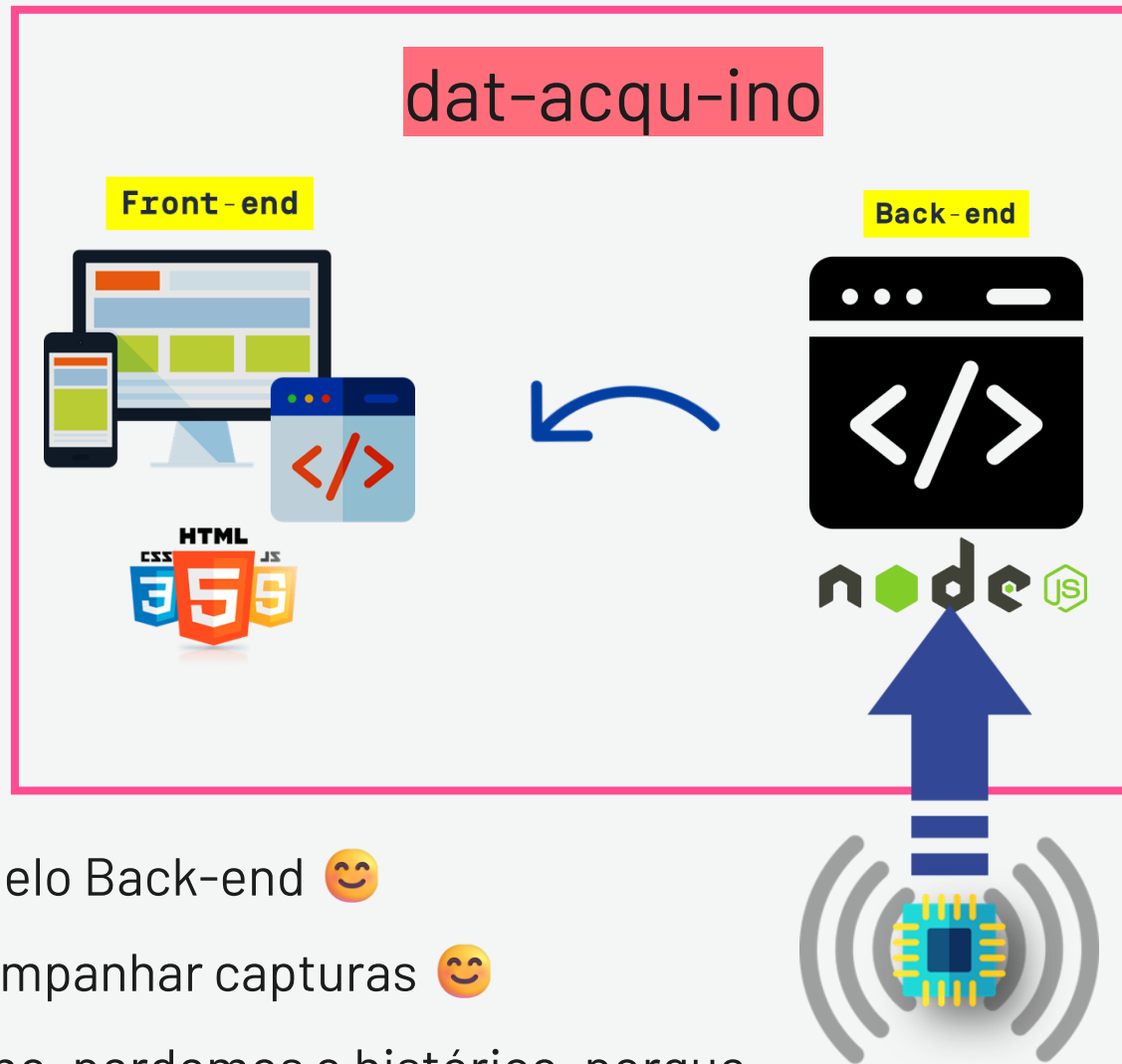
**Fizemos a ida para BD...agora a volta
para a WEB.**

Como funciona cada API?



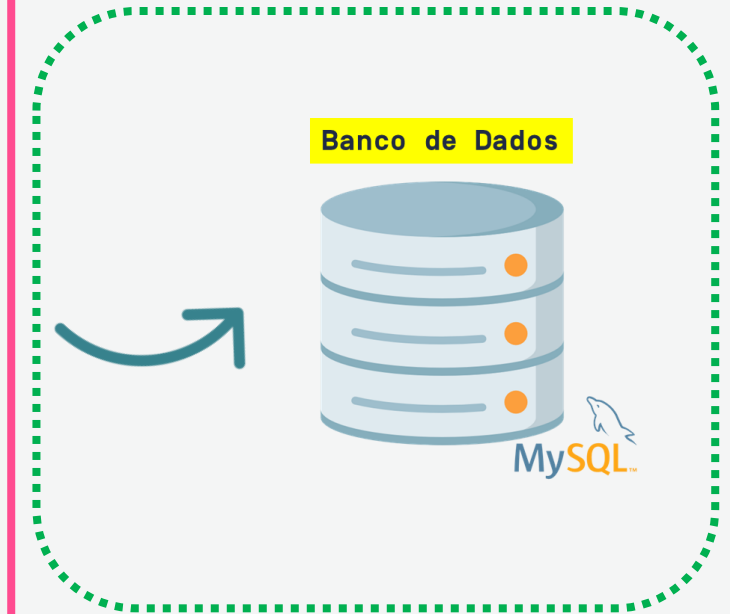
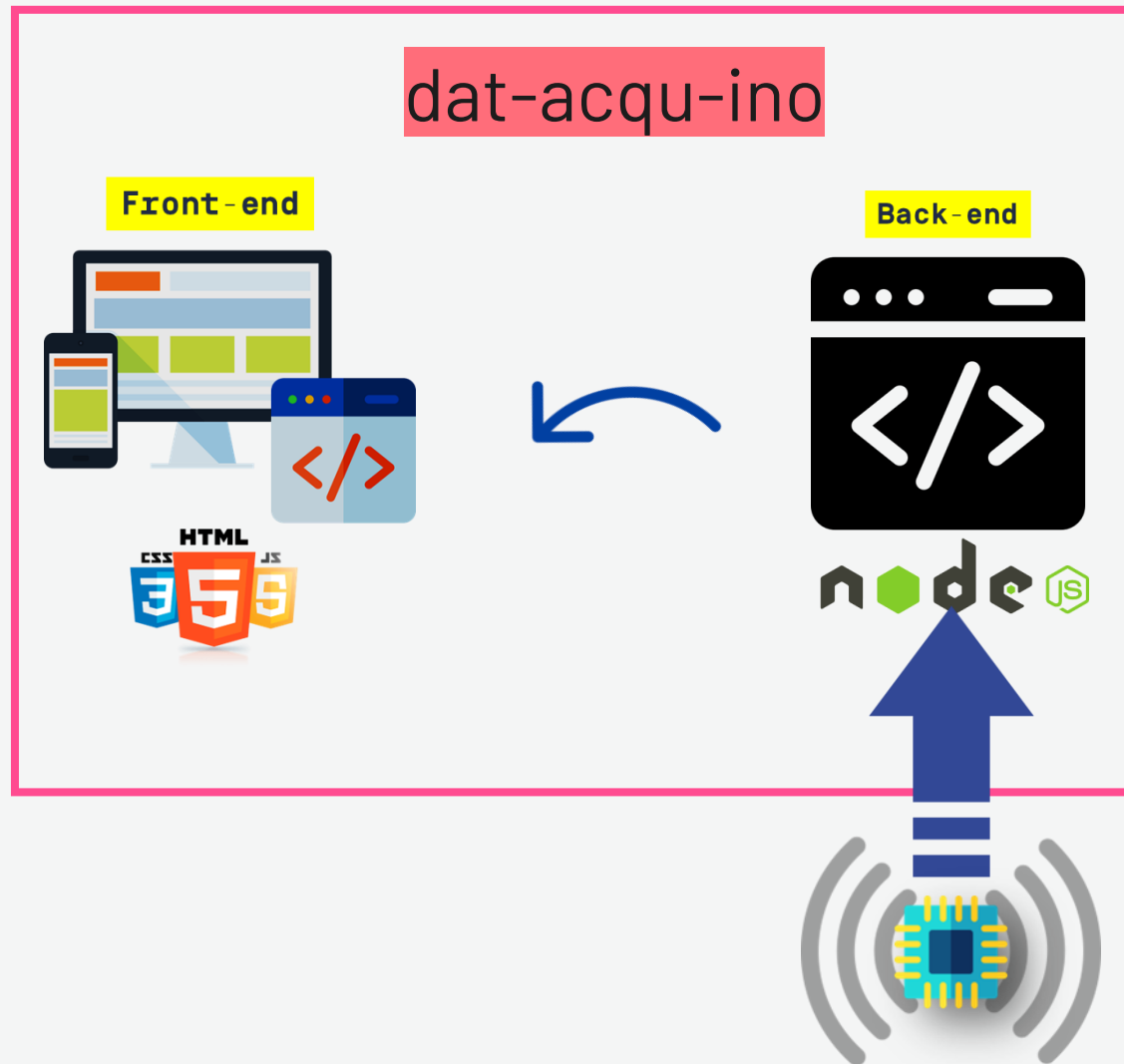
- 😊 Interação do usuário pelo Front-end 😊
- ☹️ Sem dashboards contendo dados de Arduino 😞

Como funciona cada API?



- 😊 Interação do sensor pelo Back-end 😊
- 😊 Front que ajuda a acompanhar capturas 😊
- ☹️ Quando desliga Arduino, perdemos o histórico, porque não “salva”, ou seja, não persiste no banco de dados 😞

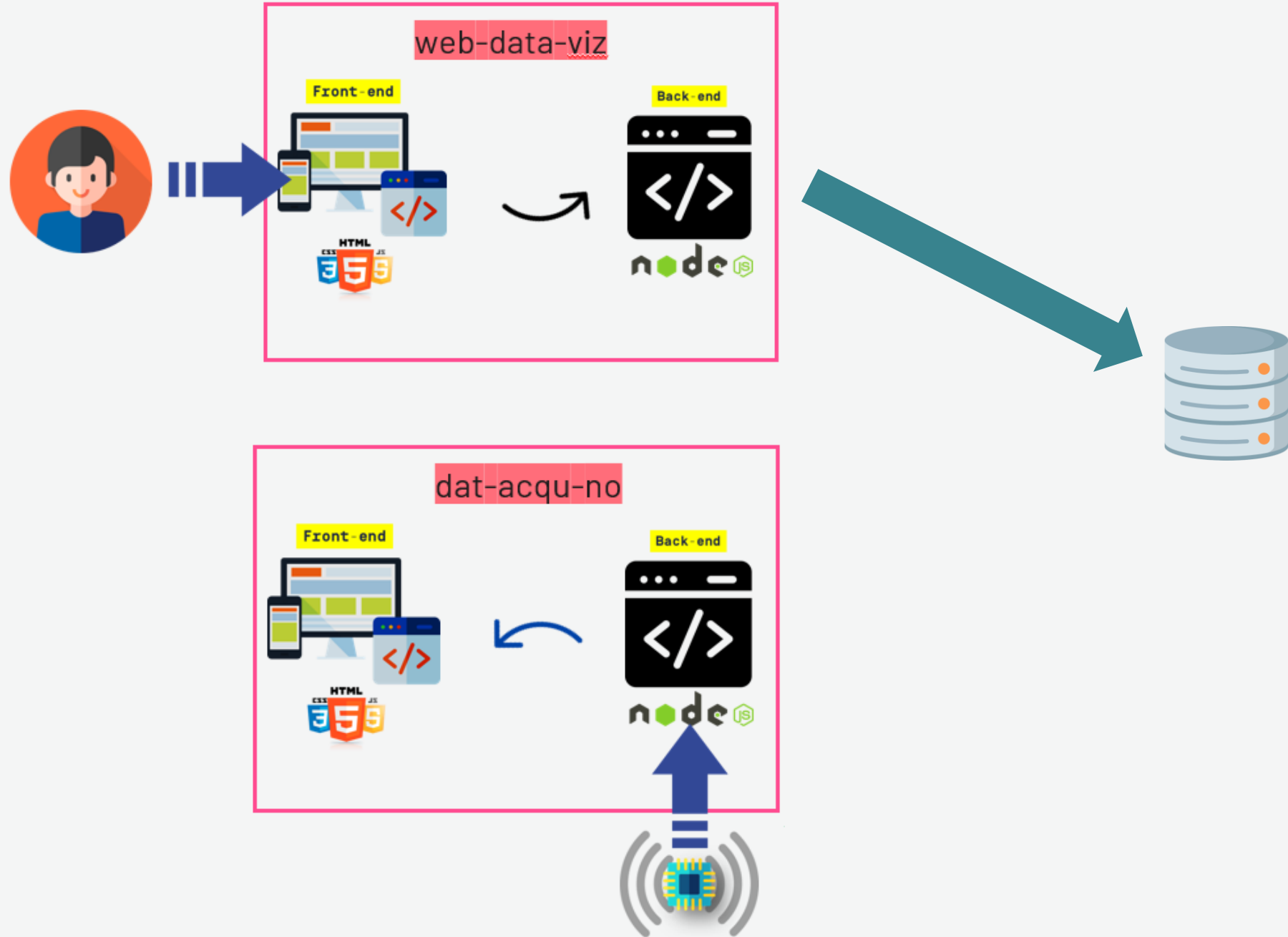
Como funciona cada API?



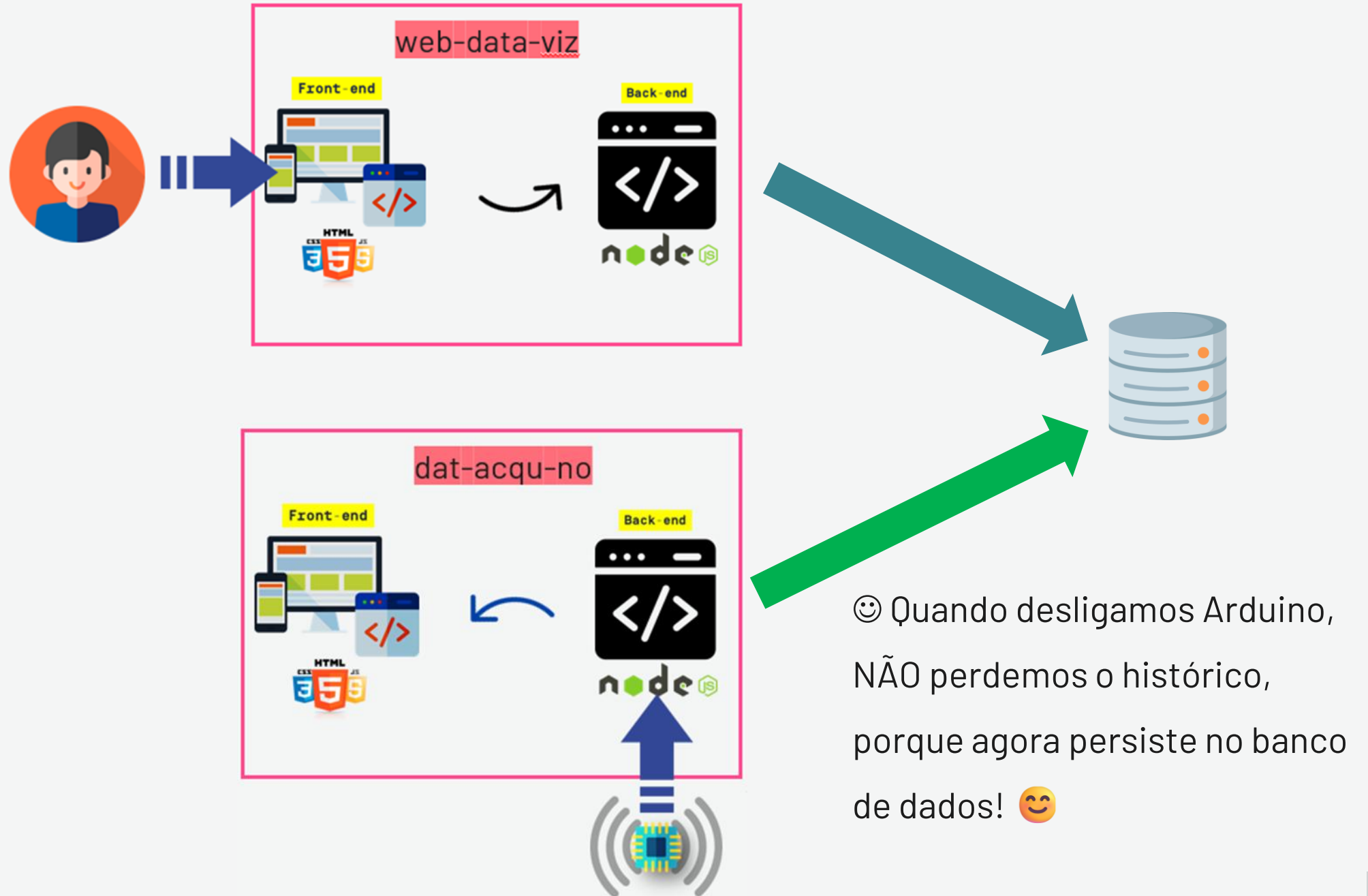
Agora
Arduino insere no Banco! 😊 😊

Vamos precisar fazer
algumas alterações
trabalhando com 2 APIs...

2 APIs

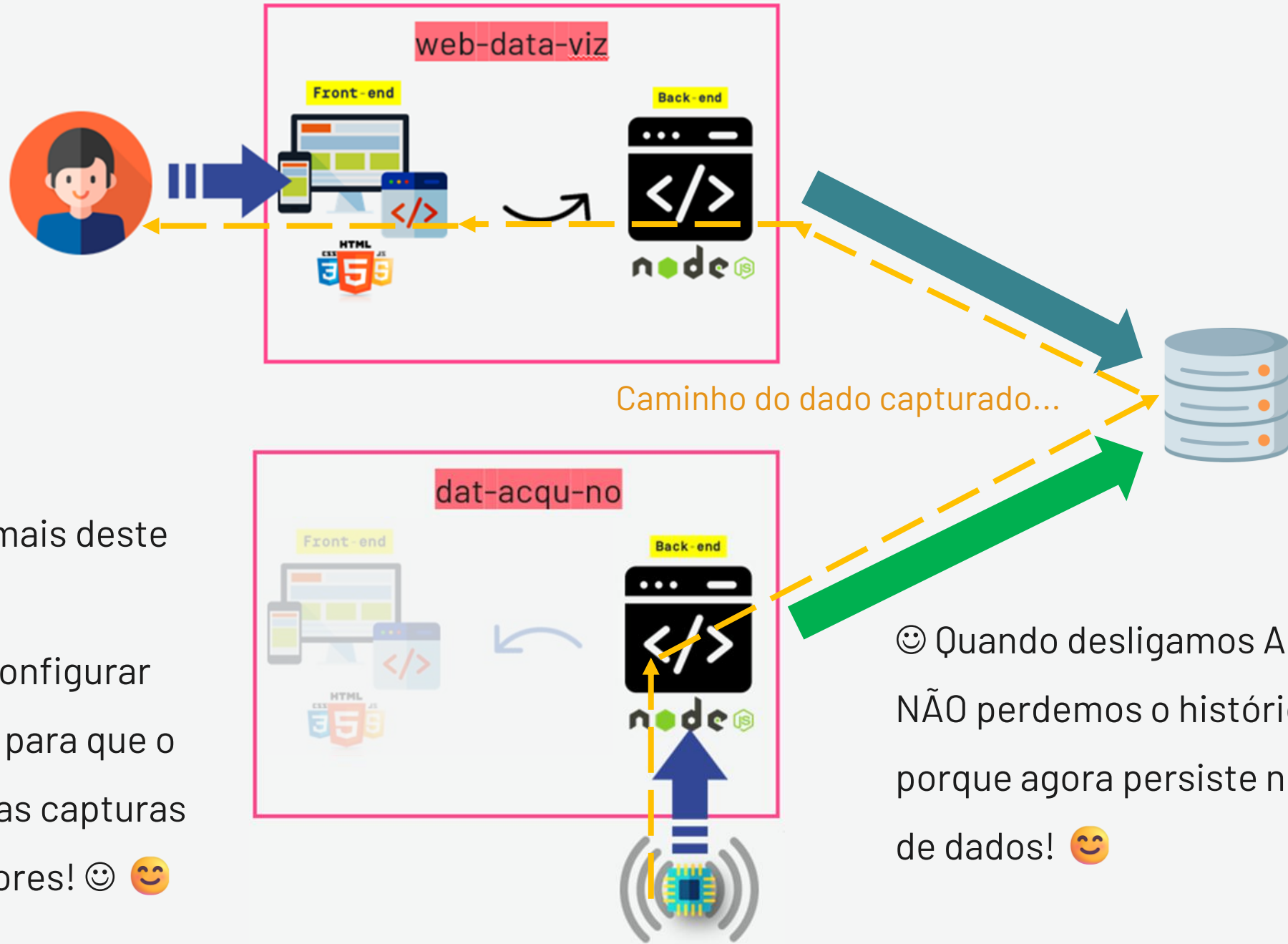


2 APIs



ATENÇÃO!!!!

2 APIs!



Não precisamos mais deste Front!

Podemos agora configurar nossa Dashboard para que o usuário visualize as capturas feitas pelos sensores! 😊 😊

😊 Quando desligamos Arduino, NÃO perdemos o histórico, porque agora persiste no banco de dados! 😊

O que eu preciso para fazer o
gráfico “funcionar”?

DADOS ! *[QUE EU CONSIGO SEM ARDUINO...]*

Vamos fazer os gráficos aparecerem...

...desafio: sem Arduino!

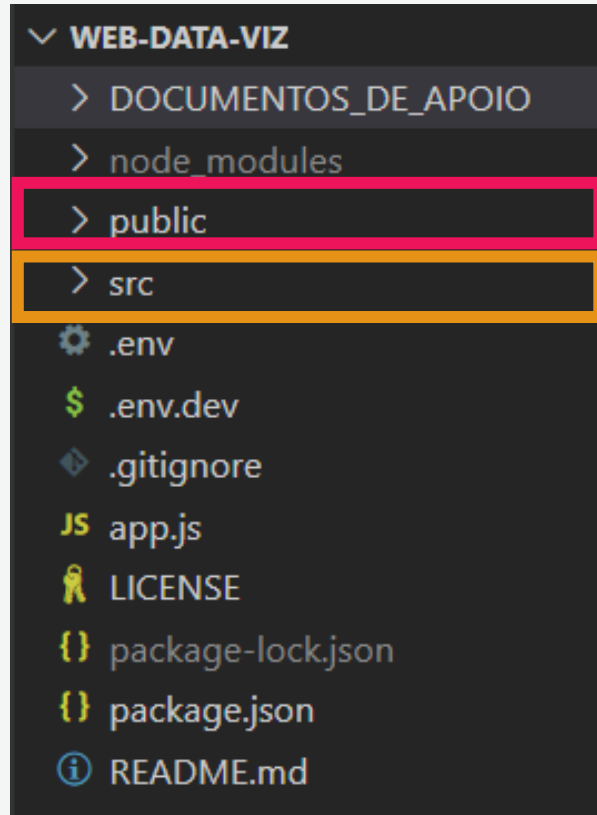
Vou fazer na minha máquina...

vocês acham que vai funcionar?

Pesquisa e Inovação

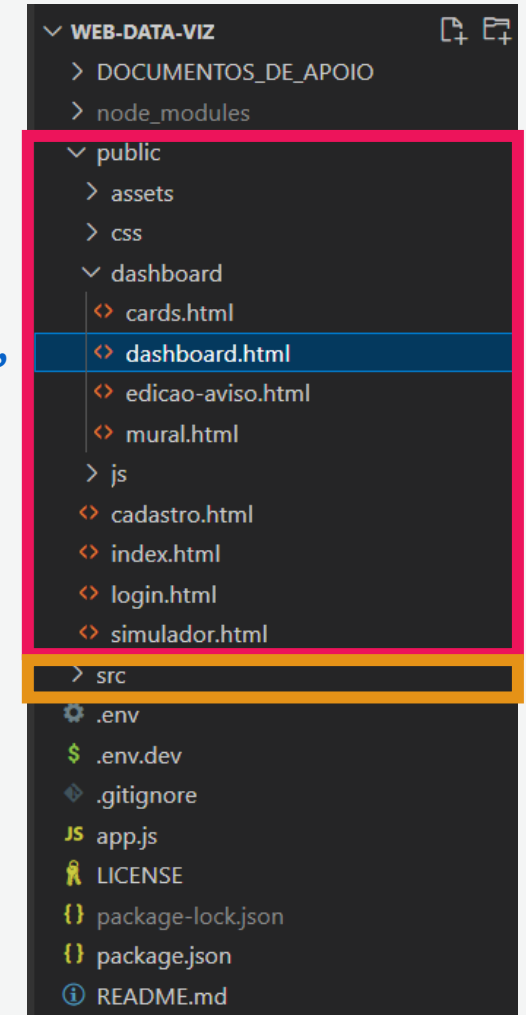
**As funções de
renderização da
Dashboard**

Dashboard



- Em /public, "Front-end"
- Em /src, "Back-end"

Vamos ver o arquivo "dashboard.html"



1. obterDadosGrafico()

```
95 function obterDadosGrafico(idAquario) {
96   if (proximaAtualizacao != undefined) {
97     clearTimeout(proximaAtualizacao);
98   }
99
100   fetch(`/medidas/ultimas/${idAquario}`)
101     .then((response) => {
102       if (response.ok) {
103         response.json().then((dados) => {
104           console.log(`Dados recebidos:`, dados);
105           plotarGrafico(dados);
106         });
107       } else {
108         console.error('Nenhum dado encontrado');
109       }
110     })
111     .catch((error) => {
112       console.error('Erro na obtenção dos dados:', error);
113     });
114 }
115
```

- Traz dados do Back-end para montar o gráfico da 1ª vez
- Invoca função 2

2. plotarGrafico()

```
120 function plotarGrafico(dados) {
121   console.log('iniciando plotagem');
122
123   var ctx = canvas_grafico.getContext('2d');
124
125   for (i = 0; i < dados.length; i++) {
126     console.log(JSON.stringify(dados[i]));
127   }
128
129   var ctx = canvas_grafico.getContext('2d');
130   window.grafico_linha = Chart.js(ctx, {
131     data: dados,
132     options: {
133       responsive: true,
134       animation: { duration: 1000 },
135       hoverMode: 'index',
136       stacked: false,
137       title: {
138         display: true,
139       },
140       scales: {
141         y: {
142           ticks: {
143             display: true,
144           },
145         },
146       },
147     },
148   });
149
150   setTimeout(() => atualizarGrafico(), 2000);
151 }
152
```

- Monta o gráfico com dados recebidos
- Invoca função 3

3. atualizarGrafico()

```
203 function atualizarGrafico(idAquario, dados) {
204   fetch(`/medidas/tempo-real/${idAquario}`)
205     .then((response) => {
206       if (response.ok) {
207         response.json().then((novoRegistro) => {
208           console.log(`Dados recebidos:`, novoRegistro);
209           console.log(`Dados atuais do gráfico:`, dados);
210
211           // tirando e colocando valores no array
212           dados.labels.shift(); // apagar o primeiro elemento
213           dados.labels.push(novoRegistro[0].tempo);
214
215           dados.datasets[0].data.shift();
216           dados.datasets[0].data.push(novoRegistro[0].valor);
217
218           dados.datasets[1].data.shift();
219           dados.datasets[1].data.push(novoRegistro[1].valor);
220
221           window.grafico_linha.update();
222
223           // Altere aqui o valor em ms se quiser
224           proximaAtualizacao = setTimeout(() => atualizarGrafico(idAquario, dados), 2000);
225         });
226       } else {
227         console.error('Nenhum dado encontrado');
228         // Altere aqui o valor em ms se quiser
229         proximaAtualizacao = setTimeout(() => atualizarGrafico(idAquario, dados), 2000);
230       }
231     })
232     .catch((error) => {
233       console.error('Erro na obtenção dos dados:', error);
234     });
235 }
236
237
```

- Traz dado mais recente e atualiza gráfico
- Invoca função 3 (recursividade)



1 - obterDadosGrafico()

```
105 function obterDadosGrafico(idAquario) {
106   if (proximaAtualizacao != undefined) {
107     clearTimeout(proximaAtualizacao);
108   }
109   fetch(`/medidas/ultimas/${idAquario}`, { cache: 'no-store' }).then
110   (function (response) {
111     if (response.ok) {
112       response.json().then(function (resposta) {
113         console.log(`Dados recebidos: ${JSON.stringify(resposta)} `)
114         ;
115         resposta.reverse();
116         plotarGrafico(resposta, idAquario);
117       });
118     } else {
119       console.error('Nenhum dado encontrado ou erro na API');
120     }
121   })
122   .catch(function (error) {
123     console.error(`Erro na obtenção dos dados p/ gráfico: ${error.
124       message}`);
125   });
126 }
```

- Traz dados do Backend para montar o gráfico da 1ª vez
- Invoca função 2
- **Dica!**

Repare que temos o fetch!

(Fetch API utilizada para possibilitar que seu projeto interaja com endpoints...
Lembra do routes/controller/models?)

2 - plotarGrafico()

```
120 function plotarGrafico(resposta, idAquario) {
121     console.log('iniciando plotagem do gráfico...');
122
123     > var dados = { ...
144
145     > for (i = 0; i < resposta.length; i++) { ...
151
152     console.log(JSON.stringify(dados));
153
154     var ctx = canvas_grafico.getContext('2d');
155     window.grafico_linha = Chart.Line(ctx, {
156         data: dados,
157         options: {
158             responsive: true,
159             animation: { duration: 500 },
160             hoverMode: 'index',
161             stacked: false,
162         > title: { ...
166         > scales: { ...
193     }
194     });
195
196     setTimeout(() => atualizarGrafico(idAquario, dados),
197         2000);
198 }
```

- Monta o gráfico com dados recebidos
- Invoca função 3

3 - atualizarGrafico()

```
203 function atualizarGrafico(idAquario, dados) {
204
205     fetch(`/medidas/tempo-real/${idAquario}`, { cache: 'no-cache' })
206     .then(response => {
207         if (response.ok) {
208             response.json().then(function (novoRegistro) {
209
210                 console.log(`Dados recebidos: ${JSON.stringify(novoRegistro)}`);
211                 console.log(`Dados atuais do gráfico: ${dados}`);
212
213                 // tirando e colocando valores no gráfico
214                 dados.labels.shift(); // apagar o primeiro
215                 dados.labels.push(novoRegistro[0].momento, novoRegistro[0].valor);
216
217                 dados.datasets[0].data.shift(); // apagar o primeiro
218                 dados.datasets[0].data.push(novoRegistro[0].valor);
219
220                 dados.datasets[1].data.shift(); // apagar o primeiro
221                 dados.datasets[1].data.push(novoRegistro[0].valor);
222
223                 window.grafico_linha.update();
224
225                 // Altere aqui o valor em ms se quiser que o gráfico seja atualizado mais rápido
226                 proximaAtualizacao = setTimeout(() => atualizarGrafico(idAquario, dados), 1000);
227             });
228         } else {
229             console.error('Nenhum dado encontrado ou erro na obtenção dos dados');
230             // Altere aqui o valor em ms se quiser que o gráfico seja atualizado mais rápido
231             proximaAtualizacao = setTimeout(() => atualizarGrafico(idAquario, dados), 1000);
232         }
233     })
234     .catch(function (error) {
235         console.error(`Erro na obtenção dos dados p/ gráfico: ${error}`);
236     });
237 }
```

- Traz dado mais recente e atualiza gráfico
- Invoca função 3 (recursividade)
- **Dica!**
Repare que temos o **fetch**!

1. obterDadosGrafico()

```
95 function obterDadosGrafico(idAquario) {
96   if (proximaAtualizacao != undefined) {
97     clearTimeout(proximaAtualizacao);
98   }
99
100   fetch(`/medidas/ultimas/${idAquario}`)
101     .then((response) => {
102       if (response.ok) {
103         response.json().then((dados) => {
104           console.log(`Dados recebidos:`, dados);
105           plotarGrafico(dados);
106         });
107       } else {
108         console.error('Nenhum dado encontrado');
109       }
110     })
111     .catch((error) => {
112       console.error('Erro na obtenção dos dados:', error);
113     });
114 }
115
```

- Traz dados do Back-end para montar o gráfico da 1ª vez
- Invoca função 2

2. plotarGrafico()

```
120 function plotarGrafico(dados) {
121   console.log('iniciando plotagem');
122
123   var ctx = canvas_grafico.getContext('2d');
124
125   for (i = 0; i < dados.length; i++) {
126     console.log(JSON.stringify(dados[i]));
127   }
128
129   var ctx = canvas_grafico.getContext('2d');
130   window.grafico_linha = Chart(ctx, {
131     data: dados,
132     options: {
133       responsive: true,
134       animation: { duration: 1000, easing: 'linear' },
135       hoverMode: 'index',
136       stacked: false,
137       title: {
138         display: true,
139         text: 'Gráfico de Medidas em Tempo Real',
140       },
141       scales: {
142         y: {
143           title: 'Medidas',
144           ticks: {
145             display: true,
146             labels: ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'],
147           },
148         },
149       },
150     },
151   });
152 }
153
```

- Monta o gráfico com dados recebidos
- Invoca função 3

3. atualizarGrafico()

```
203 function atualizarGrafico(idAquario, dados) {
204   fetch(`/medidas/tempo-real/${idAquario}`)
205     .then((response) => {
206       if (response.ok) {
207         response.json().then((novoRegistro) => {
208           console.log(`Dados recebidos:`, novoRegistro);
209           console.log(`Dados atuais do gráfico:`, dados);
210
211           // tirando e colocando valores no array
212           dados.labels.shift(); // apagar o primeiro elemento
213           dados.labels.push(novoRegistro[0].tempo);
214
215           dados.datasets[0].data.shift();
216           dados.datasets[0].data.push(novoRegistro[0].medida);
217
218           dados.datasets[1].data.shift();
219           dados.datasets[1].data.push(novoRegistro[1].medida);
220
221           window.grafico_linha.update();
222
223           // Altere aqui o valor em ms se quiser atualizar mais rápido
224           proximaAtualizacao = setTimeout(() => atualizarGrafico(idAquario, dados), 1000);
225         });
226       } else {
227         console.error('Nenhum dado encontrado');
228         // Altere aqui o valor em ms se quiser atualizar mais rápido
229         proximaAtualizacao = setTimeout(() => atualizarGrafico(idAquario, dados), 1000);
230       }
231     })
232     .catch((error) => {
233       console.error('Erro na obtenção dos dados:', error);
234     });
235 }
236
```

- Traz dado mais recente e atualiza gráfico
- Invoca função 3 (recursividade)

Agradeço
a sua atenção!



SÃO
PAULO
TECH
SCHOOL